Algorithm Engineering

Ordered Clause, Collapse Clause, Reductions, Barriers, and Storage Attributes

Mark Blacher

Friedrich Schiller University Jena

Winter Semester of 2026/27

How to Stop a Parallel For Loop

- Unfortunately, **you can't *easily* just break out** of a **parallel for loop** in OpenMP
- But there's a **workaround**

Don't Stop a Parallel For Loop, Just Avoid Work

continue_hack.cpp

```cpp
bool is_solution(int number) {  // test if number solves the problem
  for (volatile int i = 10000000; i--;) {}  // mock computation
  return number > 133 && number < 140;
}

int main() {
  constexpr int biggest_possible_number = 10000;
  // To avoid undefined code, it is necessary to use std::atomic for variables
  // where race conditions may happen.
  // https://databasearchitects.blogspot.com/2020/10/c-concurrency-model-on-x86-for-dummies.html
  atomic<bool> solution_found(false);  // if true, then we found the solution
  atomic<int> final_solution(INT32_MAX);
  const double start = omp_get_wtime();

#pragma omp parallel for schedule(dynamic)  // start parallel region
  for (int i = 0; i < biggest_possible_number; ++i) {
    if (solution_found)  // we found the solution, just continue iterating
      continue;
    if (is_solution(i)) {     // find some solution, not necessary the smallest
      solution_found = true;
      final_solution = i;
    }
  }  // end parallel region

  cout << "The solution is: " << final_solution << endl;
  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

Finding Smallest Solution by Using a Lock

continue_hack_min_solution_lock.cpp

```cpp
bool is_solution(int number) { // test if number solves the problem
  for (volatile int i = 10000000; i--;) {} // mock computation
  return number > 133 && number < 140;
}

int main() {
  constexpr int biggest_possible_number = 10000;
  atomic<int> final_solution(INT32_MAX);
  const double start = omp_get_wtime();
  mutex m; // mutual exclusion construct for updating final_solution

#pragma omp parallel for schedule(dynamic) // we could also use schedule(static, 1)
  for (int i = 0; i < biggest_possible_number; ++i) {
    if (final_solution < i) // we found a smaller solution, just continue iterating
      continue;
    if (is_solution(i)) {    // one solution found, update final_solution if it's smaller
      lock_guard<mutex> lg(m); // constructs a lock_guard, locking the given mutex
      final_solution = min(i, final_solution.load()); // only one thread computes min
    }
  }
  // end parallel region

  cout << "The smallest solution is: " << final_solution << endl;
  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

The ordered Clause

Different threads execute concurrently until they encounter the **ordered region**, which is then **executed sequentially** in the **same order as** it would get executed in a **serial loop**.

ordered.cpp

```cpp
int main() { // find smallest solution with the ordered clause
  constexpr int biggest_possible_number = 10000;
  atomic<bool> solution_found(false); // if true, then we found the solution
  int final_solution = INT32_MAX;
  const double start = omp_get_wtime();
#pragma omp parallel for ordered schedule(dynamic) // start parallel region
  for (int i = 0; i < biggest_possible_number; ++i) {
    if (solution_found) // we found the solution, just continue iterating
      continue;
    if (is_solution(i)) {
#pragma omp ordered // ordered region
      if (!solution_found) { // ordered execution of if statement
        solution_found = true; // no race condition
        final_solution = i;
      }
    }
  } // end parallel region
  cout << "The solution is: " << final_solution << endl;
  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

Nested For Loops

We can **parallelize nested for loops** with the `collapse` clause.

collapse.cpp

```
1  int main() {
2    const int N = 10;
3    const int M = 12;
4
5    /**
6     * will form a single loop of length N * M = 120
7     * and then parallelize that
8     *
9     * can be useful for balancing the work
10    */
11 #pragma omp parallel for collapse(2)
12   for (int i = 0; i < N; i++) {
13     for (int j = 0; j < M; j++) {
14       // do useful work with i and j
15     }
16   }
17 }
```

Collapse Clause Example

grid_search.cpp

```cpp
// mock k-nearest neighbors classification
void knn(int k, const string &weight, const string &metric) {
  // larger k needs more runtime
  for (volatile int i = 0; i < 10000000 * k; ++i) {}
  // computing accuracy of classification is omitted here
#pragma omp critical // output which thread did what
  cout << "k: " << k << ", weight: " << weight << ", metric: " << metric
       << ", computed with thread: " << omp_get_thread_num() << endl;
}

int main() {
  vector<int> ks{1, 3, 5, 7, 9, 11};
  vector<string> weights{"uniform", "distance"};
  vector<string> metrics{"euclidean", "manhattan"};
  const double start = omp_get_wtime();
  // parallel grid search for tuning the hyperparameters in knn
#pragma omp parallel for collapse(3) schedule(dynamic)
  for (uint64_t i = 0; i < ks.size(); ++i)
    for (uint64_t j = 0; j < weights.size(); ++j)
      for (uint64_t k = 0; k < metrics.size(); ++k)
        knn(ks[i], weights[j], metrics[k]);

  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

Reduction

Reduction is an **associative** and **commutative operation**. It is used in parallel programming to **reduce many values** into a **single result**.

In OpenMP the reduction clause looks like this:
`reduction(op:list)`

op can be `+`, `*`, `-`, `min`, `max`, `&`, `|`, `^`, `&&` and `||`

list contains the variables, separated by commas, that are to be reduced

Reduction Example

reduction.cpp

```cpp
int main() {
  int n = 100000000;
  // create a vector of size n, all values 1.0
  // long double is "usually" a 128 bit float data type
  // L indicates that 1.0 is a long double literal
  vector<double> vec(n, 1.0);
  double sum = 0.0;

  const double start = omp_get_wtime();

#pragma omp parallel for reduction(+ : sum)
  for (int i = 0; i < n; ++i) {
    sum += vec[i]; // sum up all values of vec
  }

  cout << fixed << "sum: " << sum << endl;
  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

How Reductions Work Internally

- A local copy of each `list` variable is made and initialized depending on the `op` (0 for `+`)
- Updates occur on the local copy
- Local copies are reduced into a single value and combined with the original global value

Synchronization With **Barriers**

Barrier

A barrier means that **any thread must stop** at **this point** and **cannot proceed until all other threads reach this barrier**.

barrier.cpp

```
1  int main() {
2  #pragma omp parallel
3    {
4      stringstream info;
5      info << "Hello from thread " << omp_get_thread_num() << endl;
6      cout << info.str();
7  #pragma omp barrier // move on as soon as all threads printed Hello ...
8      info.str(""); // "clear" stringstream variable
9      info << "Goodbye from thread " << omp_get_thread_num() << endl;
10     cout << info.str();
11   }
12 }
```

Implicit Barriers in OpenMP

The `for`, `sections`, `single` and `parallel` constructs in OpenMP have an **implicit barrier**.

Implicit barrier means that **there is a barrier without you explicitly positioning it there**.

In the constructs `for`, `sections` and `single` you can remove the implicit barrier with the `nowait` clause.

You **can't remove the implicit barrier** in the `parallel` construct.

Sections

sections.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
#pragma omp parallel
  {
#pragma omp sections // add nowait to remove the implicit barrier
    { // Each section is executed once by one of the threads in the team
#pragma omp section // some thread executes this block of code
      { cout << "Hello from section 1\n"; }
#pragma omp section // some thread executes this block of code
      { cout << "Hello from section 2\n"; }
#pragma omp section // some thread executes this block of code
      { cout << "Hello from section 3\n"; }
    } // after the sections construct is an implicit barrier
    cout << "Hello after the sections\n"; // this line is executed by
                                          // every thread in the team
  }
}
```

Single

The **single construct** specifies that the **associated structured block** is **executed by only one of the threads** in the team (not necessarily the master thread).

single.cpp

```
1  #include <iostream>
2  #include <omp.h>
3
4  using namespace std;
5
6  int main() {
7  #pragma omp parallel
8    {
9  #pragma omp single // only one thread executes the code in the single
10     {
11       cout << "from single: " << omp_get_thread_num() << "\n";
12     } // implicit barrier after single
13     cout << "after single\n";
14   };
15 }
```

Useful **Runtime Library Routines**

These are the **routines you already know**:

- `omp_set_num_threads(int)` *// setting desired number of threads in the parallel region*
- `omp_get_num_threads()` *// number of threads*
- `omp_get_thread_num()` *// thread id*
- `omp_get_wtime()` *// wall-clock time in seconds*

Here are **some more routines**:

- `omp_get_num_procs()` *// number of logical cores*
- `omp_get_max_threads()` *// maximum number of threads in a parallel region*
- `omp_in_parallel()` *// true if called inside a parallel region*

Library Routines Demo

library_routines.cpp

```cpp
int main() {
  cout << "num_threads: " << omp_get_num_threads() << "\n"; // prints 1
  // but, if we call omp_get_num_threads in a parallel region
  // then we get the number of threads that are executed inside the parallel region
#pragma omp parallel
#pragma omp single
  cout << "num_threads: " << omp_get_num_threads() << "\n\n"; // e.g 4

  // maximum number of threads in a parallel region (e.g. 4)
  cout << "max_threads: " << omp_get_max_threads() << "\n"; // prints 4
  omp_set_num_threads(2); // change amount of threads in a parallel region
  cout << "max_threads: " << omp_get_max_threads() << "\n\n"; // now it's 2

  // prints 4, if my machine has 4 logical cores
  cout << "num_procs: " << omp_get_num_procs() << "\n\n";
  // logical cores are the number of physical cores times the number
  // of threads (called "hyperthreads") that can run on each core

  cout << "in_parallel: " << omp_in_parallel() << "\n"; // prints 0 ("false")
  // if we call omp_in_parallel in a parallel region then we get 1 ("true")
#pragma omp parallel
#pragma omp single
  cout << "in_parallel: " << omp_in_parallel() << "\n";
}
```

Storage Attributes

Storage attributes determine how **variables defined outside the parallel region** are **handled** ("stored") **inside the parallel region**.

```
// simplified explanation but enough for our purposes
```

Useful storage attributes:

- `shared // this is default, the variable is shared between threads`
- `private // create uninitialized copy of the variable for each thread`
- `firstprivate // create initialized one-to-one copy of the variable for each thread`

Storage Attributes Demo

private_vs_firstprivate.cpp

```cpp
int main() {
  int x = 10;
  omp_set_num_threads(4);

  cout << "private(x): ";
#pragma omp parallel private(x)
  { // x is an uninitialized local copy in each thread
    x += omp_get_thread_num(); // (probably) 0 + thread_num
#pragma omp critical
    cout << x << " "; // (probably) something like: 3 2 1 0
  }

  cout << "\nfirstprivate(x): ";
#pragma omp parallel firstprivate(x)
  { // x is an initialized local one-to-one copy in each thread
    x += omp_get_thread_num(); // 10 + thread_num
#pragma omp critical
    cout << x << " "; // something like: 10 12 11 13
  }

  cout << "\n'global' x: " << x << endl; // x = 10 (global x didn't change)
}
```

Storage Attributes Remarks

- Pragmas like this one:

  `#pragma omp parallel default(none) shared(a, b) firstprivate(x, my_vector)`

  are common
- `default(none)` is good for debugging, since you must consciously specify the storage attribute for each 'global' variable used within the parallel region
- Be careful if you use `private(my_vector)`, each thread inside the parallel region creates a vector object of size=0

segmentation_fault.cpp

```
1 int main() {
2   vector<int> my_vector{1, 2, 3};
3 #pragma omp parallel default(none) shared(cout) private(my_vector)
4   {
5     // program crashes since my_vector is uninitialized (size = 0)
6     cout << my_vector[0]; // segmentation fault
7   }
8 }
```



Coding Warmup

1. **Parallelize** the **serial** π **program** (`pi_numerical_integration.cpp`) from the first lecture by **adding** only **one extra line** of code.
2. The program on page 21 generates and saves an image of the mandelbrot set. **Reduce** the **runtime** of the program **without degrading** the **image quality**.

Compile the programs with at least the following **flags**:

```
-Ofast -std=c++11 -march=native -fopenmp
```

Generating an Image

mandelbrot.cpp

```
1  int main() { // generate mandelbrot pgm (portable graymap)
2    const string image_name = "mandelbrot.pgm";
3    remove(image_name.c_str()); // remove file from disk
4    const double start = omp_get_wtime();
5    ofstream image(image_name); // file output stream
6    if (image.is_open()) {
7      image << "P2\n" << width << " " << height << " 255\n"; // pgm header
8      for (int i = 0; i < height; i++) {
9        for (int j = 0; j < width; j++) {
10         image << compute_pixel(j, i) << "\n"; // write pixel value
11       }
12     }
13     image.close(); // close file output stream
14   } else {
15     cout << "Could not open the file!";
16   }
17   cout << omp_get_wtime() - start << " seconds" << endl;
18 }
```

Possible Solution for the Warmup Image Exercise

mandelbrot_tuned.cpp

```
1  int main() { // generate mandelbrot pgm (portable graymap)
2    const string image_name = "mandelbrot.pgm";
3    remove(image_name.c_str()); // remove file from disk
4    const double start = omp_get_wtime();
5    vector<string> look_up{256}; // look-up table for strings
6    for (int i = 0; i < 256; ++i) { look_up[i] = to_string(i) + "\n"; } // fill look-up table
7    ofstream image(image_name); // file output stream
8    if (image.is_open()) {
9      image << "P2\n" << width << " " << height << " 255\n"; // write pgm header
10 #pragma omp parallel // start parallel region
11     {
12       string buffer; // each thread has it's own string buffer
13       buffer.reserve(width * 4); // reserve enough space to avoid reallocation
14 #pragma omp for schedule(dynamic) ordered // we could also use schedule(static, 1)
15       for (int i = 0; i < height; i++) {
16         buffer.clear(); // clear content of buffer
17         for (int j = 0; j < width; j++) {
18           buffer += look_up[compute_pixel(j, i)]; // fill buffer
19         }
20 #pragma omp ordered // write rows of image in sequential order
21         image << buffer; // write string buffer to file
22       }
23     } // end parallel region
24     image.close(); // close file output stream
25   } else {
26     cout << "Could not open the file!";
27   }
28   cout << omp_get_wtime() - start << " seconds" << endl;
29 }
```

Exam Assignments

- **Read** Chapter 3 titled *Programming Multi-core and Shared Memory Multiprocessors Using OpenMP* (pp. 47–78) in *Introduction to Parallel Computing* (Trobec et al., 2018).[1]
  - When run with one thread per logical core, threads from **Listing 3.1** display their numbers **randomly**, while those from **Listing 3.2** show them in the **same order**. **Explain why**.
  - What do you think, **does** the **code snippet** in **Listing 3.10** use the **cores efficiently**, why yes, why no. Consider also the variable `size` in your argumentation.
  - **From Lecture 1**, take your parallelized version of `pi_monte_carlo.cpp` and **modify** it based on the insights from *Example 3.5 on computing π using random shooting*, particularly the **random number generation** method. **Report** the **changes** you do and their **impact on performance**.
- Do the **coding warmup** on **slide 20**. (optional)

[1] You can access the content within the university network. From home, you can use VPN.